Transformer

↳ 46 ukrytych komórek

2 Attention (Single-Head)

Przekształca każde słowo (wektor) w nowe reprezentacje używając trzech macierzy wag: W_Q, W_K, W_V (to co wcześniej)

↳ 24 ukryte komórki

Zadania

Zadanie 1

Liczymy attention:

1. Utwórz wektory:

- Q = [1, 0]
- K1 = [1, 1]
- K2 = [0, 1]
- V1 = [2, 0]
- V2 = [0, 2]

2. Oblicz podobieństwo - score1 i score2: Q * K^T
3. Skalujemy - scaled1 i scaled2: dzielenie przez sqrt(d_k), uznajmy że d_k = 2
4. zastosuj softmax - probs

(Podpowiedź: To już nie będzie pojedynczy numerek a wektor: `scaled_scores = np.array([scaled1, scaled2])` i na tym robimy softmax)

5. Suma ważona Values - output
6. Odpowiedz na pytanie zawarte w print - które słowo (Key) miało największy wpływ na wynik?

```
# 0) Dane
Q = np.array([1., 0.])
K1 = np.array([1., 1.])
K2 = np.array([0., 1.])
V1 = np.array([2., 0.])
V2 = np.array([0., 2.])


# 1) podobieństwa
scores = np.array([np.dot(Q, K1), np.dot(Q, K2)])

# 2) scaled scores
scaled_scores = scores / math.sqrt(2)
print("Scaled scores:", scaled_scores)

# 3) softmax
def softmax(x):

    e_x = np.exp(x - np.max(x))
    return e_x / np.sum(e_x)

probs = softmax(scaled_scores)

# 4) weighted sum of V
output = probs[0] * V1 + probs[1] * V2

max_key_index = np.argmax(probs) + 1  # +1 bo indeks zaczyna się 0

# Print zadanie 1
print("Scores:", scores)
print("Scaled scores:", scaled_scores)
print("Softmax probabilities:", probs)
print("Output attention (wektor):", output)
```

```
print(f"Interpretacja: Największy wpływ na wynik miało słowo K{max_key_index} z prawdopodobieństwem {probs[max_key_index-1]}.")
```

```
Scaled scores: [0.70710678 0.        ]
Scores: [1. 0.]
Scaled scores: [0.70710678 0.        ]
Softmax probabilities: [0.66976155 0.33023845]
Output attention (wektor): [1.3395231 0.6604769]
Interpretacja: # TODO

Interpretacja: Największy wpływ na wynik miało słowo K1 z prawdopodobieństwem 0.6697615493266569.
```

Zadanie 2

Utwórz prostą implementację multi-head attention dla dwóch głów, korzystając z z Zadania 1.

Każda głowa będzie miała własne wektory Qi, Ki, Vi, maciesze Wq, Wk i Wv oraz własne wyniki softmax i output.

Na końcu połączysz wyniki obu głów (konkatenacja), aby uzyskać końcowy wektor.

Zwizualizuj wagi uwagi - forma dowolna

```
import numpy as np
import math
import matplotlib.pyplot as plt

def softmax(x):

  e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
  return e_x / np.sum(e_x, axis=-1, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):

  dk = Q.shape[-1]
  scores = np.matmul(Q, K.T) / math.sqrt(dk)
  if mask is not None:
    scores = scores + mask
  attn_weights = softmax(scores)
  output = np.matmul(attn_weights, V)
  return output, attn_weights
```

```
# Dane bazowe
K = np.array([[1., 1.], [0., 1.]])
V = np.array([[2., 0.], [0., 2.]])
Q = np.array([[1., 0.], [0.5, 1.0]])  # dwa zapytania dla dwóch głów

d_model = 2   # wymiar wejściowy
d_head = 2    # wymiar każdej głowy

# Głowa 1
Wq1 = np.array([[1., 0.], [0., 1.]])
Wk1 = np.array([[1., 0.], [0., 1.]])
Wv1 = np.array([[1., 0.], [0., 1.]])

# Głowa 2
Wq2 = np.array([[0.5, 0.2], [0.1, 0.7]])
Wk2 = np.array([[0.8, 0.1], [0.3, 0.9]])
Wv2 = np.array([[0.6, 0.4], [0.2, 0.8]])

# Połączenie - polecam do pętli będzie tak wygodniej :)
Wq_heads = [Wq1, Wq2]
Wk_heads = [Wk1, Wk2]
Wv_heads = [Wv1, Wv2]

head_outputs = []
head_weights = []

for i in range(len(Wq_heads)):
  # Liniowe przekształcenie macierzą wag
  Qi = np.matmul(Q, Wq_heads[i])
  Ki = np.matmul(K, Wk_heads[i])
  Vi = np.matmul(V, Wv_heads[i])

  # Obliczenie Attention dla danej głowy
  out_i, weights_i = scaled_dot_product_attention(Qi, Ki, Vi)

  head_outputs.append(out_i)
  head_weights.append(weights_i)

  print(f"GŁOWA {i+1}")
  print(f"Output:\n{out_i}")
```

```
  print(f"Wagi uwagi (Softmax):\n{weights_i}\n")


multi_head_output = np.concatenate(head_outputs, axis=1) # Łączymy macierze wzdłuż osi kolumn (axis=1)

print("Multi-Head Attention Output")
print(f"Kształt po konkatenacjii: {multi_head_output.shape}")
print(multi_head_output)


fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for i, weights in enumerate(head_weights):
  im = axes[i].imshow(weights, cmap='Blues', vmin=0, vmax=1)
  axes[i].set_title(f'Wagi uwagi - Głowa {i+1}')
  axes[i].set_xlabel('Klucze (K1, K2)')
  axes[i].set_ylabel('Zapytania (Q1, Q2)')
  axes[i].set_xticks([0, 1], labels=['K1', 'K2'])
  axes[i].set_yticks([0, 1], labels=['Q1', 'Q2'])

  for r in range(weights.shape[0]):
    for c in range(weights.shape[1]):
      axes[i].text( c,r,f'{weights[r, c]:.3f}')

plt.tight_layout()
plt.show()
```

```
GŁOWA 1
Output:
[[1.3395231 0.6604769]
 [1.174958  0.825042 ]]
Wagi uwagi (Softmax):
[[0.66976155 0.33023845]
 [0.587479   0.412521  ]]

GŁOWA 2
Output:
[[0.85896422 1.14103578]
 [0.85063854 1.14936146]]
Wagi uwagi (Softmax):
[[0.57370527 0.42629473]
 [0.56329817 0.43670183]]

Multi-Head Attention Output
Kształt po konkatenacjii: (2, 4)
[[1.3395231  0.6604769  0.85896422 1.14103578]
 [1.174958   0.825042   0.85063854 1.14936146]]
```



